Compositional Primitive Formation (What It *Is*)

It is the architecture that defines:

1. **What a primitive *is*** (a structural operator)
2. **When a primitive *forms*** (the preconditions)
3. **What a primitive *does*** (state updates)
4. **When a primitive *collapses*** (threshold failures)

Transformers don't have this.
Transformers *approximate* composition through statistics over tokens.

Aurora explicitly models composition as an operation.

This is the heart of the architecture.

Example Level Description (Plain but precise)

A **compositional primitive** is an explicit operator that binds conceptual states according to strict rules.

Examples:

- **WE** = binds Roles into a composite Role if mutual agency + shared intention + recognition are present
- **THEN** = binds event/state A to event/state B with temporal ordering + context inheritance
- **WHILE / UNTIL** = bind states under persistence or conditional continuation
- **IF–THEN** = conditional state binding
- **BECAUSE** = causal linkage

Each primitive contains:

- **Preconditions**
- **Activation rules**
- **Update logic**
- **Collapse conditions**

Formal Level Description (Architectural Specification)

A primitive $P$is a **typed relational operator**:

$$P: (S_1, S_2, \ldots, S_n) \to S'$$

Where:

- $S_i$are conceptual states (Roles, Domains, Spans, Events)
- $S'$is the updated structural configuration

Each primitive is defined by:

1. Activation Function

$$\text{activate}_P(S,\text{context}) \to \begin{cases} \text{instantiate } P & \text{if preconditions met} \\ \text{null} & \text{otherwise} \end{cases}$$

2. Update Rule

Defines how Roles, Domains, or Spans change:

$$\text{update}_P(S, P_{\text{instance}}) \to S'$$

3. Interpretability Dependency

Primitives rely on relational alignment (your interpretability metric):

$$I(R_1, R_2 \mid D) \geq \theta_P$$

If alignment drops below threshold, the primitive collapses.

4. Collapse Condition

$$I < \theta_P \Rightarrow \text{collapse}(P)$$

The system removes or reverts the structured operator.